

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Design Pattern II

a cura di Massimo La Morgia

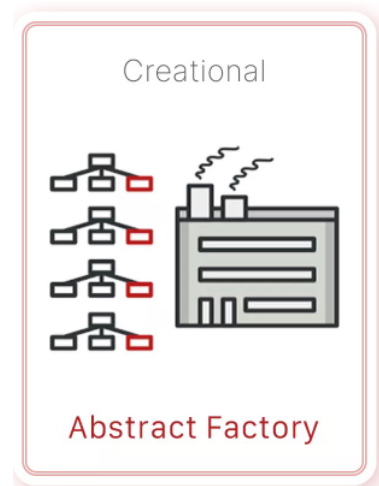


SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

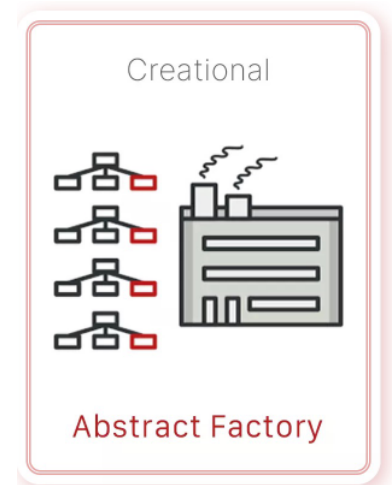
Design pattern parte II



Abstract Factory pattern

Abstract Factory pattern 1

L'Abstract Factory è un pattern di **progettazione creazionale** (come il Factory Method e il Singleton) il cui scopo è gestire la creazione di intere "famiglie" di oggetti correlati o dipendenti, senza specificare le loro classi concrete.

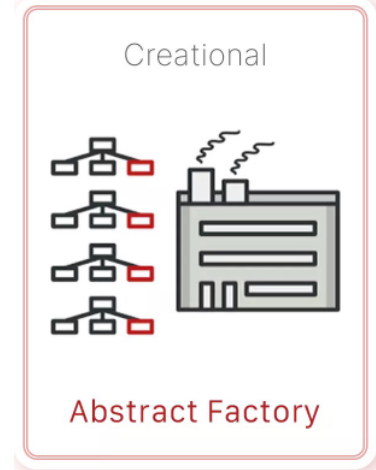


Fornisce un'astrazione (la Fabbrica Astratta) per la creazione di diversi tipi di prodotti (ad esempio, sedie, tavoli, divani), assicurando che i prodotti creati siano **compatibili tra loro** (ad esempio, tutti in stile *moderno* o tutti in stile *vittoriano*).

Abstract Factory pattern 2

💬 Scopo

Abstract Factory è un pattern di progettazione creazionale che ti permette di produrre famiglie di oggetti correlati senza specificare le loro classi concrete.



😞 Problema

Il tuo codice è costituito da classi che rappresentano:

Una famiglia di prodotti correlati, ad esempio: Chair (Sedia) + Sofa (Divano) + CoffeeTable (Tavolino da Caffè). Ad esempio, i prodotti Chair + Sofa + CoffeeTable sono disponibili in queste varianti: Modern (Moderno), Victorian (Vittoriano) e ArtDeco.

Hai bisogno di un modo per creare singoli oggetti d'arredamento in modo che si abbinino ad altri oggetti della stessa famiglia.

I clienti si arrabbiano parecchio quando ricevono arredamento non abbinato.

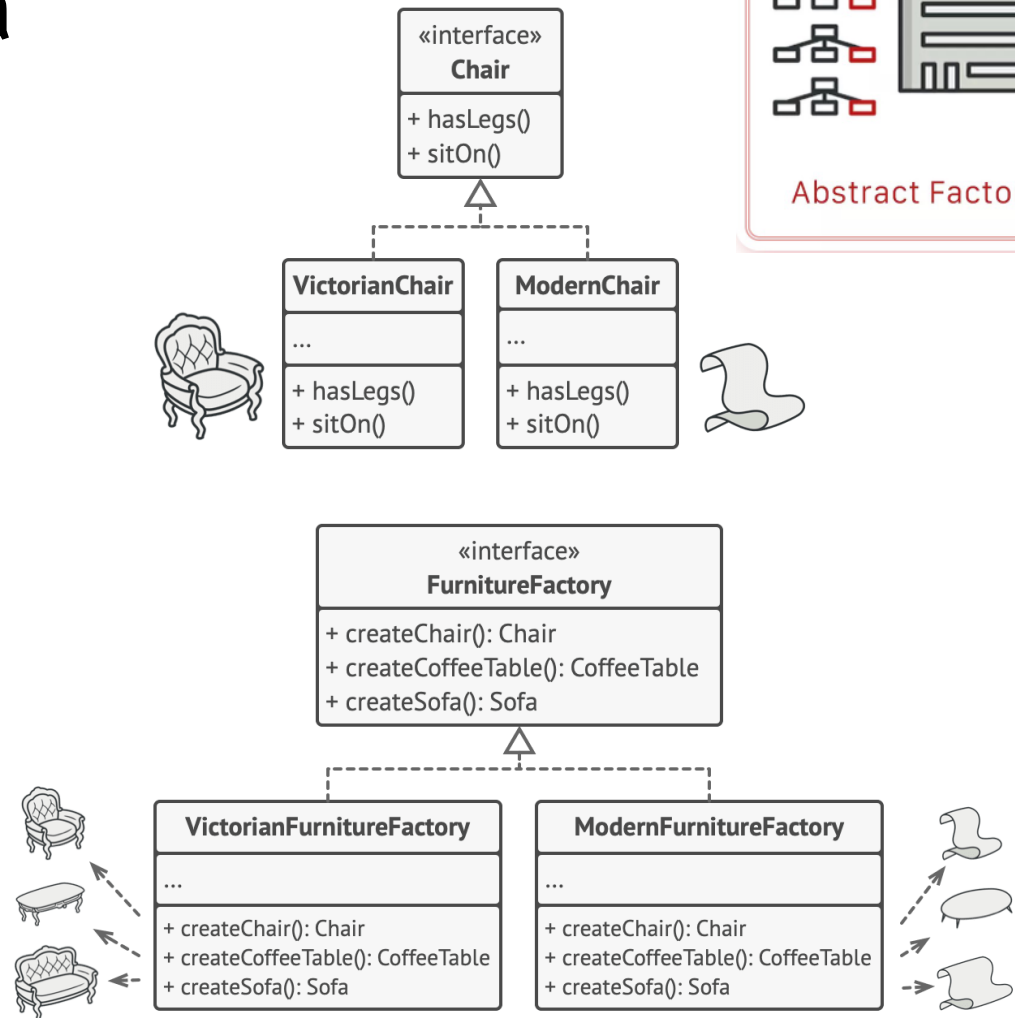
Inoltre, non vuoi modificare il codice esistente quando aggiungi nuovi prodotti o famiglie di prodotti al programma.

I fornitori di arredamento aggiornano i loro cataloghi molto spesso, e non vorresti dover cambiare il codice *core* (principale) ogni volta che ciò accade.

Abstract Factory pattern 3

😊 Soluzione / Ricetta

- 1) Definire una interfaccia per ogni prodotto.
- 2) Definire diverse varianti di ogni Prodotto in accordo con gli stili.
- 3) Definire una astrazione/interfaccia di Fabbriche.
- 4) Definire diverse concretizzazioni di Fabbriche, una per ogni stile.



Abstract Factory pattern 4

Esempio

```
public abstract class AbstractFactory {  
    abstract Sedia getSedia();  
    abstract Tavolo getTavolo();  
}
```

```
public class MobiliModerni extends AbstractFactory{  
    @Override  
    Sedia getSedia() {  
        return new SediaModerna();  
    }  
    ...  
}
```

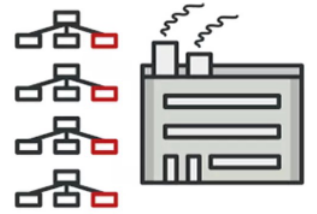
```
public class MobiliAntichi extends AbstractFactory{  
    @Override  
    Sedia getSedia() {  
        return new SediaAntica();  
    }  
    ...  
}
```

```
public interface Sedia {  
    /* */  
}
```

```
public class SediaModerna implements Sedia{  
    /* */  
}
```

```
public class SediaAntica implements Sedia{  
    /* */  
}
```

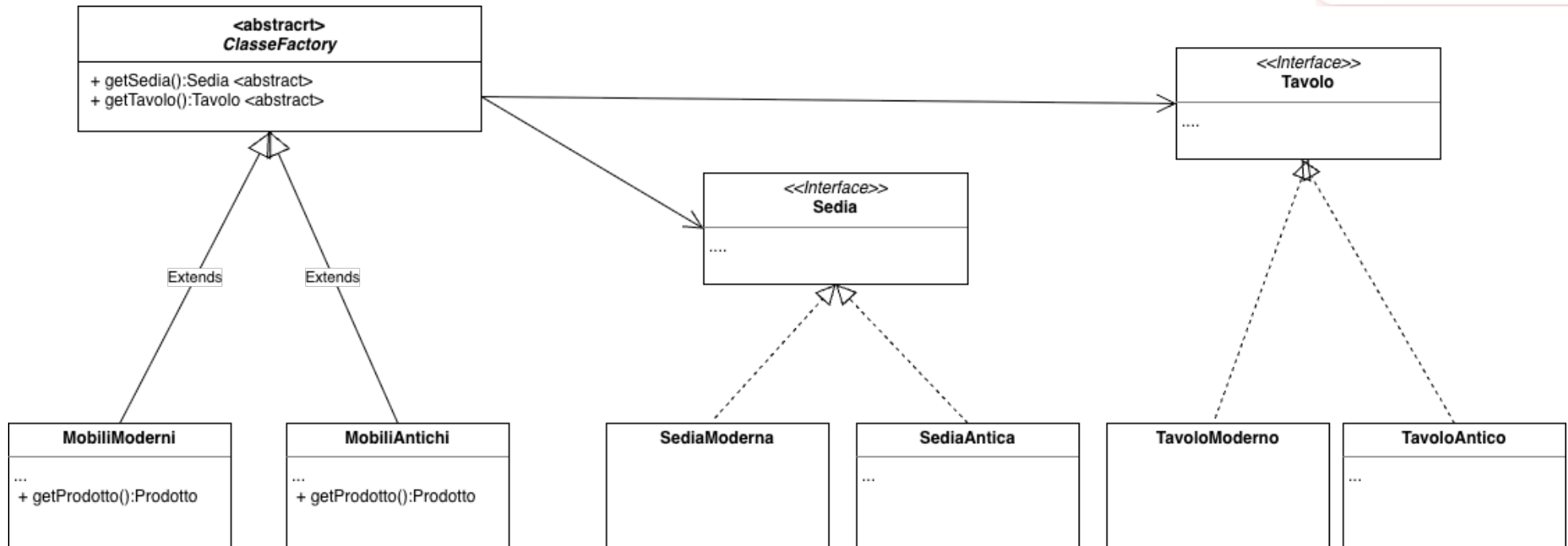
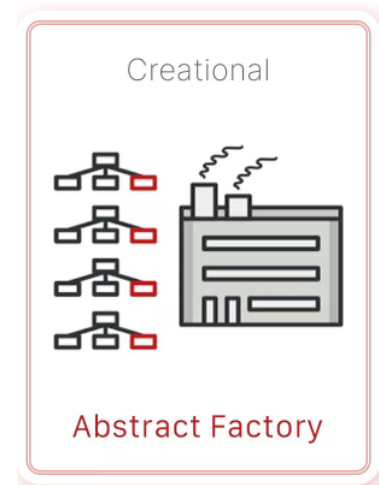
Creational



Abstract Factory

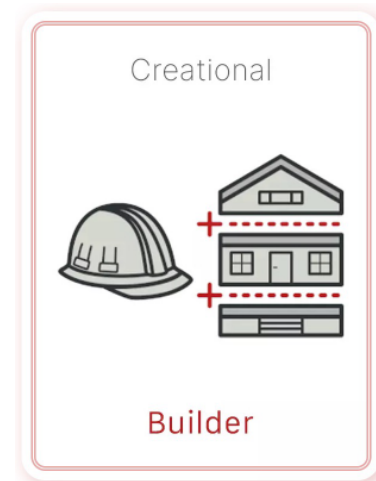
Abstract Factory pattern 5

🚗 Esempio/Schema



Posso aggiungere nel tempo:

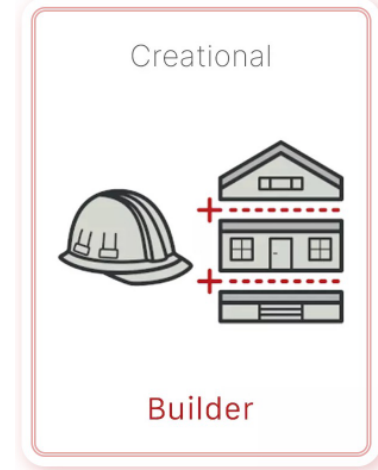
- 1) Altri fornitori
- 2) Altri prodotti
- 3) Nuovi stili



Builder pattern

Builder pattern 1

Il Builder pattern (o "Pattern del Costruttore") è un **pattern di progettazione creazionale** (come Singleton e Factory Method) che mira a risolvere il problema della costruzione di oggetti complessi.



Il Builder Pattern viene tipicamente usato per risolvere il problema dei **Costruttori Telescopici**:

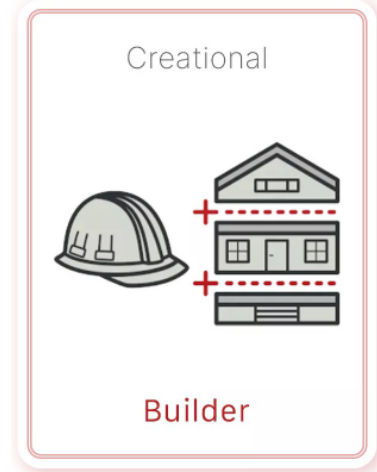
Quando una classe ha molti campi opzionali, si finisce spesso per creare costruttori con un numero crescente di parametri (new Prodotto(param1, param2), new Prodotto(param1, param2, param3, param4), ecc.).

Questo è difficile da gestire, rende il codice illeggibile e crea confusione su quale parametro corrisponda a quale campo.

Builder pattern 2

💬 Scopo

Builder è un pattern di progettazione creazionale che ti permette di costruire oggetti complessi passo dopo passo. Il pattern ti consente di produrre diversi tipi e rappresentazioni di un oggetto utilizzando lo stesso codice di costruzione.



😞 Problema

Immagina un oggetto complesso che richiede una laboriosa inizializzazione passo dopo passo di molti campi e oggetti annidati.

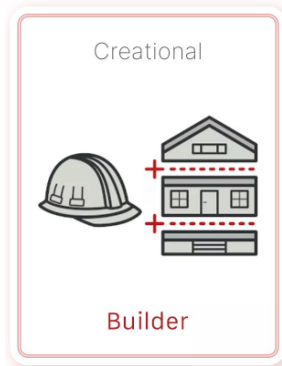
Tale codice di inizializzazione è di solito sepolto all'interno di un **costruttore mostruoso con molti parametri**.

La classe è **sovraccaricata di molteplici costruttori** che delegano la costruzione al **costruttore mostruoso** (detti costruttori telescopici).

Si capisce a colpo d'occhio che il codice della classe è ingestibile e neanche una buona documentazione aiuterà ad usare agevolmente la classe. Peggio ancora, nessuno/a vorrà mai gestire la manutenzione del codice di questa classe.

Builder pattern 3

😊 Soluzione / Ricetta

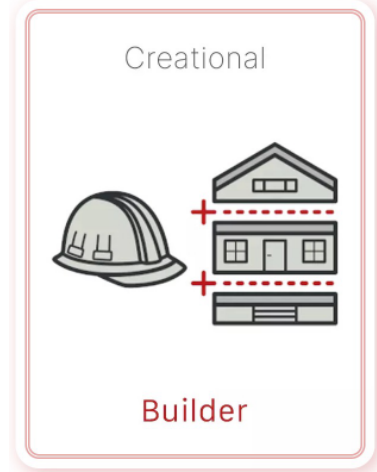


1. Crea una nuova classe, tipicamente chiamata **ClasseBuilder** se **Classe** è la classe di cui si vogliono costruire gli oggetti.
 - **ClasseBuilder** può anche essere una classe statica annidata all'interno di **Classe** (in quel caso, si chiamerà **Classe.Builder**)
 - Il costruttore di **Classe** viene impostato come privato o si usa la visibilità di package se **ClasseBuilder** non è annidato.
2. **ClasseBuilder** è dotata di metodi per l'impostazione dei valori iniziali dei campi dell'oggetto da costruire
 - Ciascuno di questi metodi restituisce l'istanza del builder (**this**), in modo che si possano effettuare chiamate in cascata, come con `append()` di `StringBuffer`.
3. **ClasseBuilder** è dotata di un metodo **build** che restituisce un'istanza di **Classe** (o di una sua sottoclasse!) secondo le impostazioni specificate.
4. **ClasseBuilder** può essere dotata di un costruttore per obbligare l'impostazione iniziale di uno o più campi dell'oggetto da costruire.

Builder pattern 4

Esempio

Costruttori
telescopici



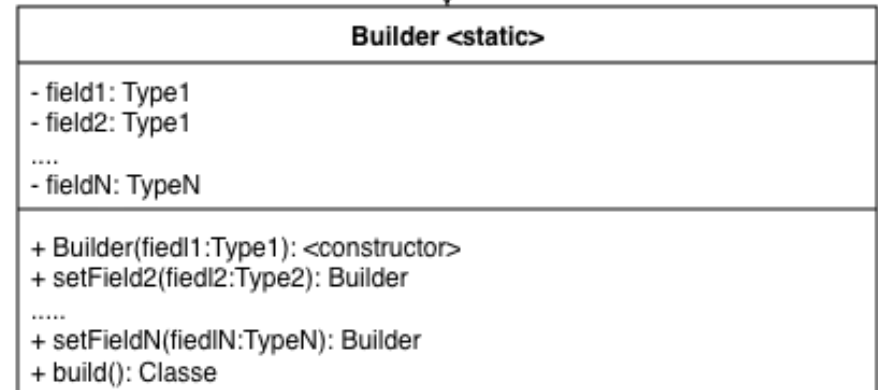
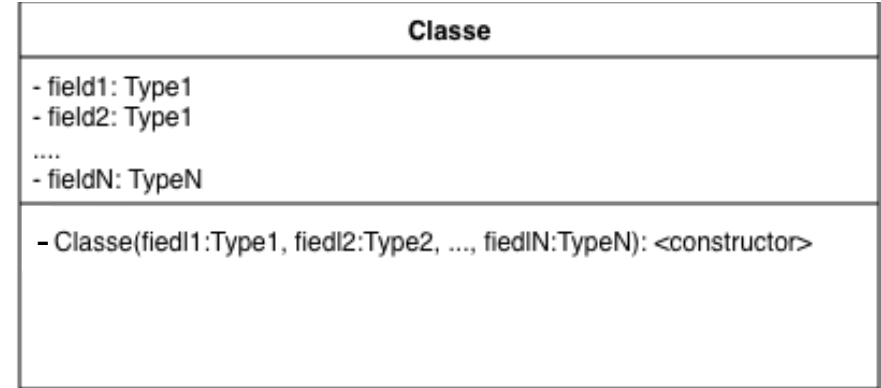
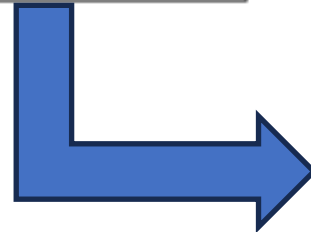
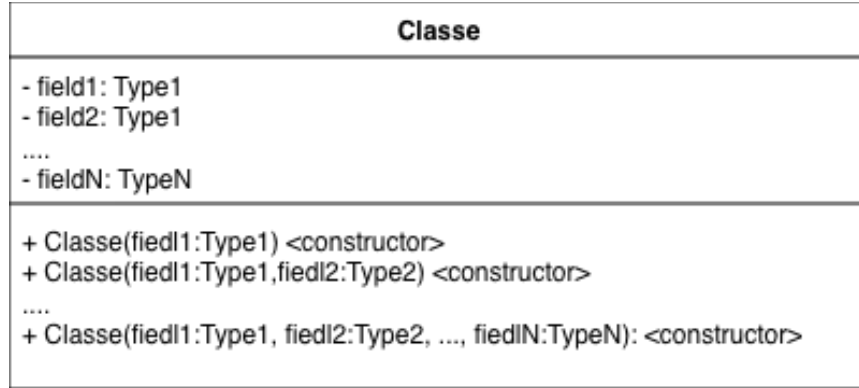
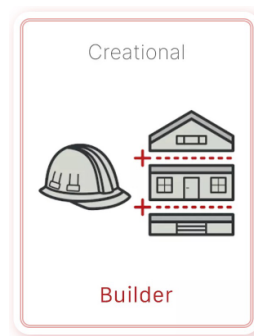
```
public class Classe {
    Type1 filed1; Type2 filed2; ... ;TypeN filedN;
    public Classe(Type1 param1, Type2 param2){
        this(param1,param2, defaultValue3 ,...,defaultValueN);
    }
    public Classe(Type1 param1, Type2 param2, ... , TypeN paramN-1){
        this(param1,param2,..., paramN-1, defaultValueN);
    }
    public Classe(Type1 param1, Type2 param2, ... , TypeN paramN) {
        ...
    }
    ...
}
```

Costruttore
«mostruoso»

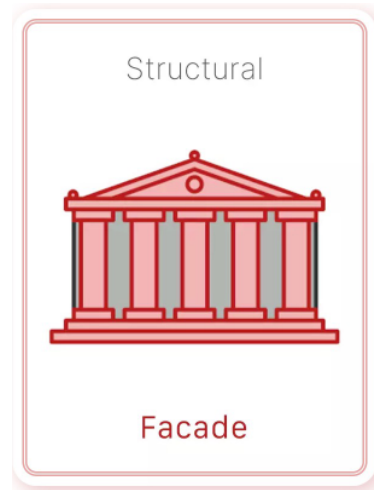
```
public class Classe {
    Type1 filed1; Type2 filed2; ... ;TypeN filedN;
    private Classe(Type1 param1, Type2 param2, ... , TypeN paramN) {
        ...
    }
    public static class Builder{
        Type1 filed1; Type2 filed2; ... ;TypeN filedN;
        public Builder(Type1 param1) { this.filed1=param1; }
        public Builder setField2(Type2 param2){ this.filed2=param2; return this;}
        ...
        public Builder setFieldN(Type2 param2){ this.filed2=param2; return this;}
        public Classe build() { return new Classe(filed1, fiels2, ... , fieldN) };
    }
    ...
}
```

Builder pattern 5

Schema



- Non ho il problema dei costruttori telescopici.
- Facilità nell'aggiunta di un campo.
- Facile da usare: si pensi alla componibilità di Stream<T> e StringBuilder.



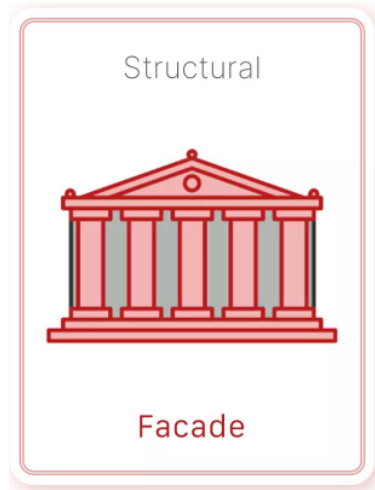
Facade pattern

Facade pattern 1

Il Facade pattern (o "Pattern Facciata") è un pattern di progettazione strutturale (a differenza dei pattern creazionali discussi in precedenza) che mira a semplificare l'interazione con un sistema complesso.

La sua funzione principale è quella di fornire un'interfaccia unificata e di alto livello a un sotto-sistema complesso composto da molte classi e interfacce.

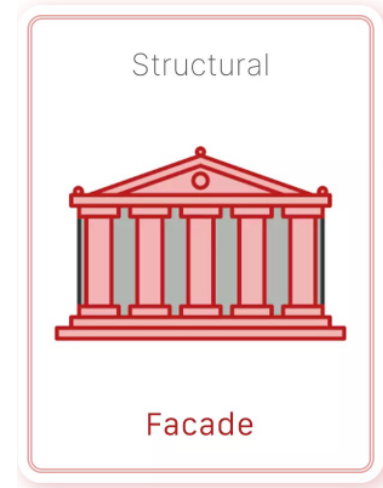
il Facade funge da **punto d'accesso unico** che maschera la complessità interna del sotto-sistema, rendendolo più facile da usare per il codice client.



Facade pattern 2

💬 Scopo

Facade è un pattern di progettazione strutturale che fornisce un'interfaccia semplificata a una libreria, un framework o qualsiasi altro insieme complesso di classi.



😞 Problema

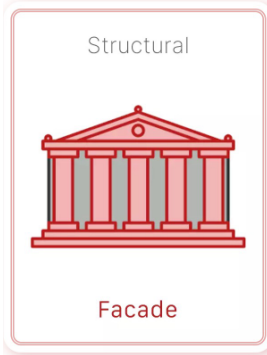
Immagina di dover far funzionare il tuo codice con un ampio insieme di oggetti che appartengono a una libreria o a un framework sofisticato.

Normalmente, dovresti inizializzare tutti questi oggetti, tenere traccia delle dipendenze, eseguire i metodi nell'ordine corretto e così via.

Di conseguenza, la logica di business delle tue classi diventerebbe strettamente accoppiata ai dettagli di implementazione delle classi di terze parti, rendendo il codice difficile da comprendere e mantenere.

Facade pattern 3

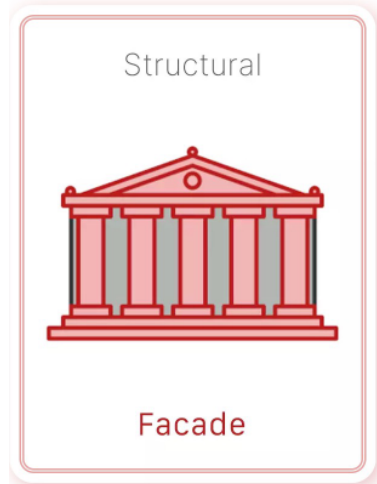
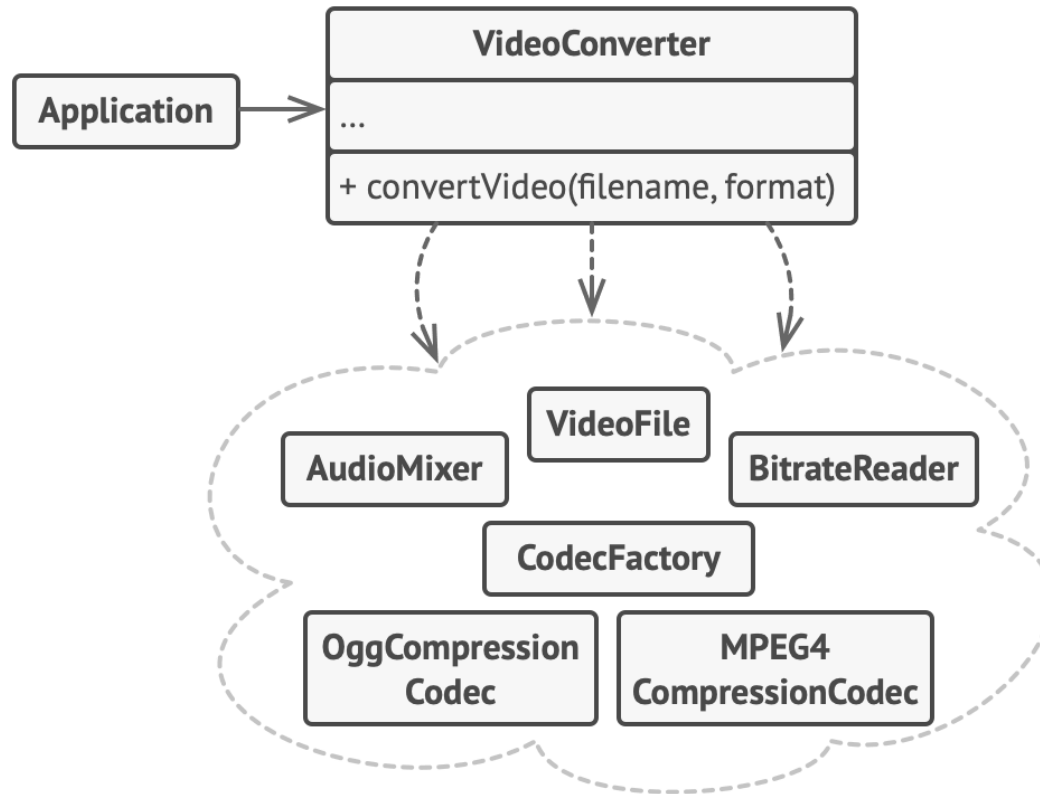
😊 Soluzione / Ricetta



- 1) Una **Facade** è una classe che fornisce un'interfaccia semplice a un sottosistema complesso che contiene molte parti dinamiche (o elementi interconnessi).
- 2) Una Facade potrebbe fornire funzionalità limitate rispetto al lavorare direttamente con il sottosistema. Tuttavia, include solo quelle funzionalità a cui i client sono realmente interessati.
- 3) Avere una Facade è utile quando il software deve interagire con una libreria sofisticata che ha decine di funzionalità, ma hai bisogno solo di una piccola parte delle sue funzionalità.
- 4) Ad esempio, un'app che carica brevi video divertenti di gatti sui social media potrebbe potenzialmente utilizzare una libreria professionale di conversione video. Tuttavia, tutto ciò di cui ha veramente bisogno è una classe con il singolo metodo
`encode(nomefile, formato)`
- 5) Dopo aver creato una classe del genere e averla collegata alla libreria di conversione video, avrai la tua prima facciata.

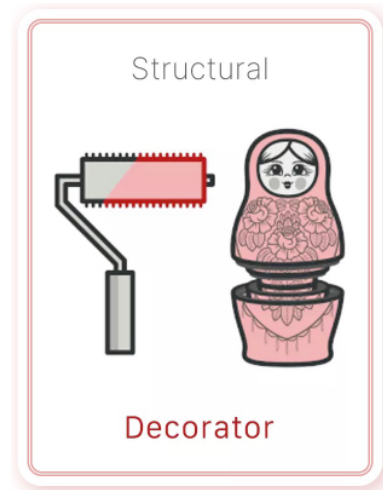
Facade pattern 4

Esempio



Invece di far lavorare il tuo codice direttamente con decine di classi del framework, crei una classe **Facade** che **incapsula** tale funzionalità e la nasconde al resto del codice.

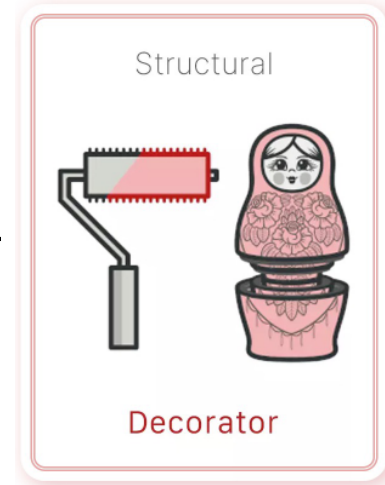
Questa struttura ti aiuta anche a minimizzare lo sforzo di aggiornare (il codice) alle future versioni del software o di sostituirlo con un altro. L'unica cosa che dovrai cambiare nella tua app sarà l'implementazione dei metodi della **Facade**.



Decorator pattern

Decorator pattern 1

Il Decorator pattern (o "Pattern Decoratore") è un pattern di progettazione strutturale (come il Facade) che fornisce un'alternativa flessibile alla sottoclassificazione per estensione delle funzionalità di un oggetto. Aggiunge nuove responsabilità e funzionalità dinamicamente a un oggetto esistente, avvolgendolo/parassitandolo con un «decoratore» (senza che l'oggetto se ne accorga).

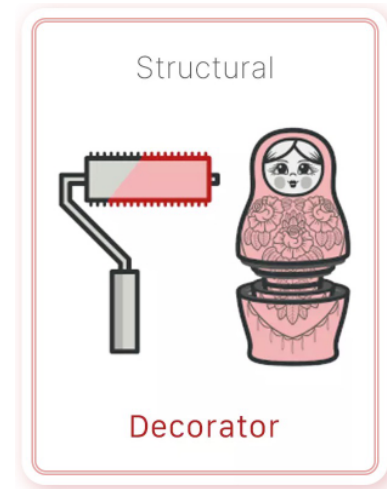


il Decorator ti permette di *decorare* un oggetto avvolgendolo in una o più classi wrapper che aggiungono comportamenti *prima* o *dopo* l'esecuzione del comportamento dell'oggetto originale.

Decorator pattern 2

💬 Scopo

Decorator è un pattern di progettazione strutturale che ti permette di allegare nuovi comportamenti agli oggetti inserendo questi oggetti all'interno di speciali oggetti *wrapper* (involucro) che contengono i comportamenti.



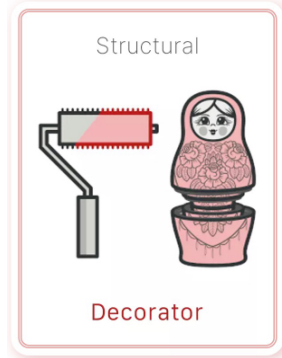
😞 Problema

Immagina di stare lavorando su una libreria di notifica che permette ad altri programmi di notificare i propri utenti in merito a eventi importanti. La versione iniziale della libreria era basata sulla classe `Notifier` (Notificatore) che aveva solo alcuni campi, un costruttore e un singolo metodo `send` (invia). Il metodo poteva accettare un argomento *messaggio* da un client e inviare il messaggio a un elenco di e-mail che venivano passate al notificatore tramite il suo costruttore.

Ad un certo punto, ti rendi conto che gli utenti della libreria si aspettano più che semplici notifiche via email. Molti di loro vorrebbero ricevere un SMS riguardo a problemi critici. Altri vorrebbero essere notificati su Facebook e, naturalmente, gli utenti aziendali adorerebbero ricevere notifiche su Slack.

Decorator pattern 3

😊 Soluzione / Ricetta

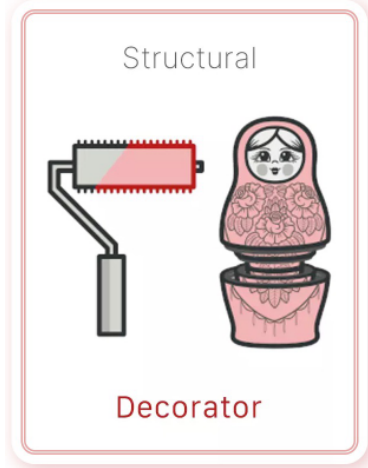
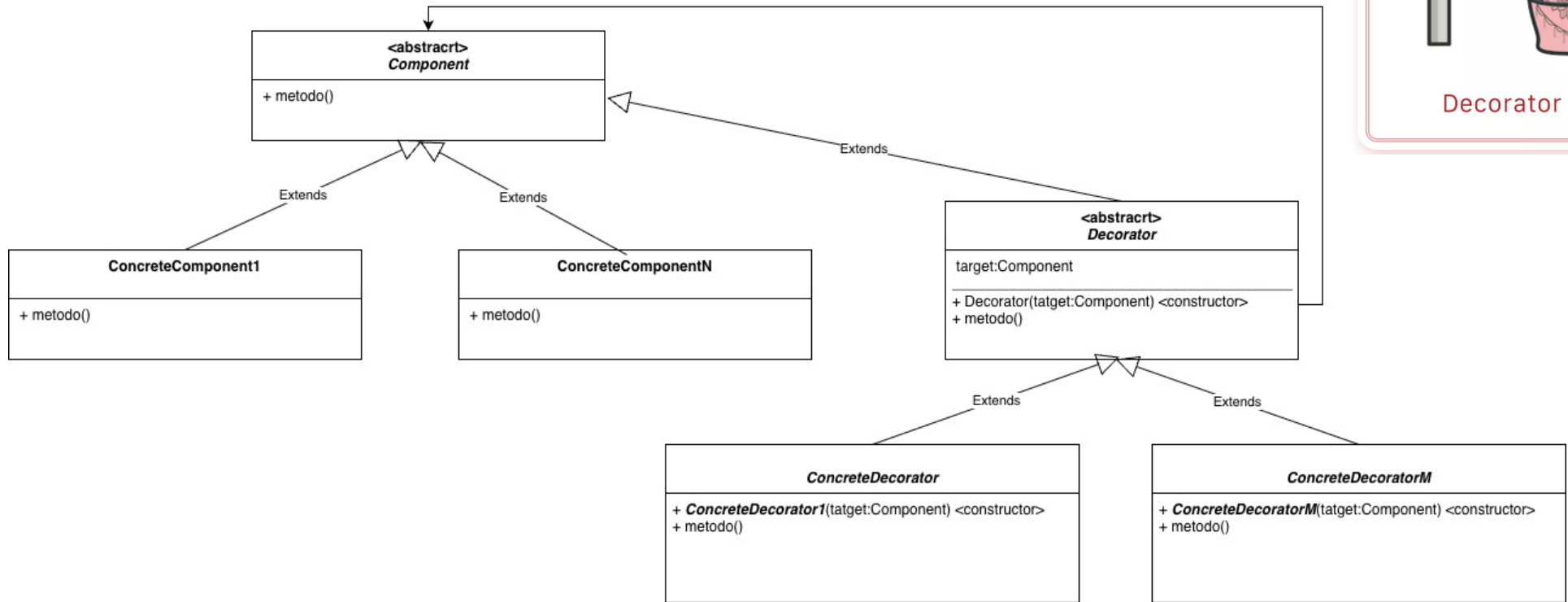


"Wrapper" (Involucro) è il soprannome alternativo per il pattern Decorator che esprime chiaramente l'idea principale del pattern.

Un *wrapper/Decorator* è un oggetto che può essere collegato a un oggetto *target/Component*. Il **Decorator** contiene lo stesso insieme di metodi del *target/Component* e gli delega tutte le richieste che riceve. Tuttavia, il **Decorator** può alterare il risultato eseguendo alcune operazioni sia prima che dopo aver passato la richiesta al *target/Component*.

Decorator pattern 4

Schema



- Si definisce chiaramente chi è il Component astratto e si definiscono le concretizzazioni di Component.
- Il Decorator è un Component anche lui
- Il Decorator si costruisce specificando il Componente target da parassitare
- I Decoratori Concreti sfruttano, nella loro implementazione di **metodo()**, l'implementazione esistente di quella dell'oggetto Componente target che parassitano/decorano.

Decoro un **oggetto a tempo di esecuzione** senza modificare/estendere la classe componente concreta.

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli e <https://design-patterns-101.vercel.app/>

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM2011O000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.